

[Gemini](#)

Eigen SFML Particle Simulation Boilerplate

Conversation with Gemini

You said

from these three file, i want to build the following feature

Create

the boilerplate script that is to become the MAIN script of the simulation.

This script should have the ability to draw a singular circle and given some initial position/velocity data of the circle, update the window at some frame rate updating the particles position based on its initial velocity. You should

see the particle move across the screen in a straight line, all while using the eigen library for c++

Gemini said

It looks like you've already got a solid foundation with Eigen and SFML 3.0. To consolidate your three files into a single, cohesive boilerplate that focuses on moving a particle using **Eigen's vector types** for position and velocity, we need to refactor the Particle class to store its data as `Eigen::Vector2f` rather than individual floats.

Using Eigen vectors allows you to perform physics calculations (like `position += velocity * dt`) in a single line of clean code.

Refactored Boilerplate Script

This version integrates the logic from your three files into a streamlined main script.

C++

```
#include <SFML/Graphics.hpp>
```

```
#include <Eigen/Core>
```

```

#include <iostream>

#include <vector>

#include <random>


// --- Particle Class Definition ---

class Particle {
public:
    Eigen::Vector2f position;
    Eigen::Vector2f velocity;
    float mass;
    sf::CircleShape shape;


    Particle(float m, sf::Color color, Eigen::Vector2f pos, Eigen::Vector2f vel)
        : mass(m), position(pos), velocity(vel) {

        // Visual representation setup
        float radius = std::sqrt(mass) * 2.0f;
        shape.setRadius(radius);
        shape.setOrigin({radius, radius}); // Center the origin for Eigen mapping
        shape.setFillColor(color);
    }


    void update(float deltaTime) {
        // Linear motion:  $P_{\text{new}} = P_{\text{old}} + V * dt$ 
        position += velocity * deltaTime;
    }
};

```

```

// Boundary bouncing logic (Window: 1300x800)
if (position.x() < 0 || position.x() > 1300) velocity.x() *= -1.0f;
if (position.y() < 0 || position.y() > 800) velocity.y() *= -1.0f;

// Sync SFML shape with Eigen position
shape.setPosition({position.x(), position.y()});
}
};

// --- Main Simulation ---
int main() {
    // 1. Setup Window
    sf::RenderWindow window(sf::VideoMode({1300, 800}), "Eigen Particle Simulation");
    window.setFramerateLimit(60);

    // 2. Initialize Randomness
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<float> posWidth(100.0f, 1200.0f);
    std::uniform_real_distribution<float> posHeight(100.0f, 700.0f);
    std::uniform_real_distribution<float> velDist(-200.0f, 200.0f);

    // 3. Create a Singular Circle (as requested)
    // Using Eigen::Vector2f for initial data
    Eigen::Vector2f startPos(posWidth(gen), posHeight(gen));
    Eigen::Vector2f startVel(velDist(gen), velDist(gen));

```

```

Particle myParticle(25.0f, sf::Color::Cyan, startPos, startVel);

sf::Clock clock;

// 4. Main Loop
while (window.isOpen()) {
    float dt = clock.restart().asSeconds();

    while (const std::optional event = window.pollEvent()) {
        if (event->is<sf::Event::Closed>())
            window.close();
    }

    // Update Physics
    myParticle.update(dt);

    // Render
    window.clear(sf::Color::Black);
    window.draw(myParticle.shape);
    window.display();
}

return 0;
}

```

Key Improvements Made:

- **Eigen Integration:** Replaced `x`, `y`, `velocityX`, and `velocityY` with `Eigen::Vector2f`. This is more idiomatic for physics simulations and makes future vector math (like gravity or collisions) much easier.
- **Vectorized Update:** The update logic now uses `position += velocity * deltaTime`, treating the position and velocity as true mathematical vectors.
- **Origin Centering:** By setting the origin to the center of the circle (`radius`, `radius`), the Eigen coordinates represent the center of the particle rather than the top-left corner.
- **SFML 3.0 Compatibility:** Used the modern `window.pollEvent()` syntax with `std::optional` as seen in your demo file.

Would you like me to add a gravity vector using Eigen so the particle follows a parabolic arc instead of a straight line?

You said

I see that code doesn't let user declare how many particles to create, create a function allow user to enter amount of particles to create and limit the particles to 100 particles

Gemini said

To allow for a user-defined number of particles while maintaining a safety cap, we can integrate a helper function that validates input against your 100-particle limit.

Based on your existing `sfml_3_0_demo.cpp` logic, we will transition from a single `Particle` object to a `std::vector<Particle>` to manage multiple instances.

Updated Simulation Script

C++

```
#include <SFML/Graphics.hpp>
```

```
#include <Eigen/Core>
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <random>
```

```
#include <algorithm> // For std::min
```

```
// --- Particle Class Definition ---
```

```

class Particle {
public:
    Eigen::Vector2f position;
    Eigen::Vector2f velocity;
    float mass;
    sf::CircleShape shape;

    // Default constructor for vector resizing
    Particle() : mass(1.0f), position(0,0), velocity(0,0) {}

    Particle(float m, sf::Color color, Eigen::Vector2f pos, Eigen::Vector2f vel)
        : mass(m), position(pos), velocity(vel) {

        float radius = std::sqrt(mass) * 2.0f; // Scale radius by mass
        shape.setRadius(radius);
        shape.setOrigin({radius, radius});
        shape.setFillColor(color);
    }

    void update(float deltaTime) {
        //  $P_{\text{new}} = P_{\text{old}} + V * dt$ 
        position += velocity * deltaTime;

        // Boundary bouncing for 1300x800 window
        if (position.x() < 0 || position.x() > 1300) velocity.x() *= -1.0f;
        if (position.y() < 0 || position.y() > 800) velocity.y() *= -1.0f;
    }
}

```

```

        shape.setPosition({position.x(), position.y()});
    }
};

// --- Helper Function for User Input ---
int getParticleCount() {
    int count;

    std::cout << "Enter the number of particles (Max 100): ";
    if (!(std::cin >> count)) return 0;

    // Limit to 100 particles as requested
    if (count > 100) {
        std::cout << "Limit exceeded. Setting count to 100." << std::endl;
        return 100;
    }
    return std::max(0, count);
}

// --- Main Simulation ---
int main() {
    int numParticles = getParticleCount();

    sf::RenderWindow window(sf::VideoMode({1300, 800}), "Eigen Multi-Particle
Simulation");

    window.setFramerateLimit(60);

```

```

// Setup Randomness

std::random_device rd;
std::mt19937 gen(rd());
std::uniform_real_distribution<float> massDist(5.0f, 100.0f);
std::uniform_real_distribution<float> posWidth(100.0f, 1200.0f);
std::uniform_real_distribution<float> posHeight(100.0f, 700.0f);
std::uniform_real_distribution<float> velDist(-150.0f, 150.0f);
std::uniform_int_distribution<int> colorDist(50, 255);


// Initialize Particle Vector

std::vector<Particle> particles;
for (int i = 0; i < numParticles; ++i) {
    Eigen::Vector2f startPos(posWidth(gen), posHeight(gen));
    Eigen::Vector2f startVel(velDist(gen), velDist(gen));
    sf::Color color(colorDist(gen), colorDist(gen), colorDist(gen));

    particles.emplace_back(massDist(gen), color, startPos, startVel);
}


sf::Clock clock;

while (window.isOpen()) {
    float dt = clock.restart().asSeconds();

    while (const std::optional event = window.pollEvent()) {

```



```

        if (event->is<sf::Event::Closed>())
            window.close();
    }

    window.clear(sf::Color::Black);

    // Update and Draw all particles
    for (auto& p : particles) {
        p.update(dt);
        window.draw(p.shape);
    }

    window.display();
}

return 0;
}

```

Key Logic Changes:

- **The getParticleCount Function:** This handles the std::cin logic and uses a simple if check to ensure the value never exceeds 100.
- **std::vector<Particle>:** Instead of one object, we use a container to store and iterate through your particles.
- **Eigen in the Loop:** Each particle is initialized with unique Eigen::Vector2f values for position and velocity inside the setup loop.
- **Enhanced Randomness:** I adjusted the color range to 50-255 to ensure the particles aren't too dark to see against the black background.

Would you like me to add a "Reset" keybind that lets you re-enter the particle count without restarting the program?

You said

Instead setting a randomness for color and mass, set color to red and mass to 40 for all particles created.

Gemini said

To update the simulation according to your requirements, I have modified the initialization loop to remove the random distributions for mass and color. Every particle will now be initialized with a mass of **40** and the color **Red**.

Updated Main Script

C++

```
#include <SFML/Graphics.hpp>
```

```
#include <Eigen/Core>
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <random>
```

```
#include <algorithm>
```

```
// --- Particle Class Definition ---
```

```
class Particle {
```

```
public:
```

```
    Eigen::Vector2f position;
```

```
    Eigen::Vector2f velocity;
```

```
    float mass;
```

```
    sf::CircleShape shape;
```

```
    Particle() : mass(1.0f), position(0,0), velocity(0,0) {}
```

```
Particle(float m, sf::Color color, Eigen::Vector2f pos, Eigen::Vector2f vel)
```

```
    : mass(m), position(pos), velocity(vel) {
```

```
        // Radius is derived from the fixed mass of 40
```

```
        float radius = std::sqrt(mass) * 2.0f;
```

```
        shape.setRadius(radius);
```

```
        shape.setOrigin({radius, radius});
```

```
        shape.setFillColor(color);
```

```
    }
```

```
void update(float deltaTime) {
```

```
    //  $P = P + V * dt$  using Eigen vectors
```

```
    position += velocity * deltaTime;
```

```
    // Boundary logic for 1300x800 window
```

```
    if (position.x() < 0 || position.x() > 1300) velocity.x() *= -1.0f;
```

```
    if (position.y() < 0 || position.y() > 800) velocity.y() *= -1.0f;
```

```
    shape.setPosition({position.x(), position.y()});
```

```
}
```

```
};
```

```
// --- Helper Function for User Input ---
```

```
int getParticleCount() {
```

```
    int count;
```

```

std::cout << "Enter the number of particles (Max 100): ";

if (!(std::cin >> count)) return 0;


if (count > 100) {
    std::cout << "Limit exceeded. Setting count to 100." << std::endl;
    return 100;
}

return std::max(0, count);
}


// --- Main Simulation ---

int main() {

    int numParticles = getParticleCount();


    sf::RenderWindow window(sf::VideoMode({1300, 800}), "Eigen Uniform Particle
Simulation");

    window.setFramerateLimit(60);


    // Setup Randomness only for Position and Velocity

    std::random_device rd;
    std::mt19937 gen(rd());

    std::uniform_real_distribution<float> posWidth(100.0f, 1200.0f);
    std::uniform_real_distribution<float> posHeight(100.0f, 700.0f);
    std::uniform_real_distribution<float> velDist(-150.0f, 150.0f);


    std::vector<Particle> particles;

```

```

for (int i = 0; i < numParticles; ++i) {
    // Position and Velocity remain unique to see movement
    Eigen::Vector2f startPos(posWidth(gen), posHeight(gen));
    Eigen::Vector2f startVel(velDist(gen), velDist(gen));

    // Parameters forced to Mass: 40, Color: Red
    particles.emplace_back(40.0f, sf::Color::Red, startPos, startVel);
}

sf::Clock clock;

while (window.isOpen()) {
    float dt = clock.restart().asSeconds();

    while (const std::optional event = window.pollEvent()) {
        if (event->is<sf::Event::Closed>())
            window.close();
    }

    window.clear(sf::Color::Black);

    for (auto& p : particles) {
        p.update(dt);
        window.draw(p.shape);
    }
}

```

```

        window.display();
    }

    return 0;
}

```

Changes implemented:

- **Fixed Mass:** The `massDist(gen)` call was replaced with a literal `40.0f`.
- **Fixed Color:** The `sf::Color` object is now explicitly `sf::Color::Red` instead of using random RGB values.
- **Visual Consistency:** Since mass is tied to the radius calculation (), all particles will now appear exactly the same size on screen.
- **Maintained Eigen Motion:** The particles still use `Eigen::Vector2f` for their position and velocity updates to ensure they move in straight lines across the window.

Would you like to see how to implement a basic collision check so these identical red particles bounce off each other?

You said

set a timeout for the program after 5 minutes to terminate the program

Gemini said

To implement a 5-minute timeout, we can track the total elapsed time using an `sf::Clock` or an `sf::Time` accumulator within the main loop. Once the total time exceeds 300 seconds, we simply break out of the loop and terminate the program.

The following script integrates the requested timeout, the fixed mass/color settings, and the Eigen-based particle logic.

Updated Main Script with 5-Minute Timeout

```

C++

#include <SFML/Graphics.hpp>

#include <Eigen/Core>

#include <iostream>

```

```
#include <vector>

#include <random>

#include <algorithm>


class Particle {
public:
    Eigen::Vector2f position;
    Eigen::Vector2f velocity;
    float mass;
    sf::CircleShape shape;


    Particle() : mass(40.0f), position(0, 0), velocity(0, 0) {}

    Particle(float m, sf::Color color, Eigen::Vector2f pos, Eigen::Vector2f vel)
        : mass(m), position(pos), velocity(vel) {

        float radius = std::sqrt(mass) * 2.0f;
        shape.setRadius(radius);
        shape.setOrigin({radius, radius});
        shape.setFillColor(color);
    }

    void update(float deltaTime) {
        position += velocity * deltaTime;

        // Boundary bouncing logic
```

```

    if (position.x() < 0 || position.x() > 1300) velocity.x() *= -1.0f;
    if (position.y() < 0 || position.y() > 800) velocity.y() *= -1.0f;

    shape.setPosition({position.x(), position.y()});
}
};

```

```

int getParticleCount() {
    int count;

    std::cout << "Enter the number of particles (Max 100): ";
    if (!(std::cin >> count)) return 0;

    if (count > 100) {
        std::cout << "Limit exceeded. Setting count to 100." << std::endl;
        return 100;
    }

    return std::max(0, count);
}

```

```

int main() {
    int numParticles = getParticleCount();

    sf::RenderWindow window(sf::VideoMode({1300, 800}), "Eigen Simulation - 5 Min
Timeout");

    window.setFramerateLimit(60);

```



```

std::random_device rd;
std::mt19937 gen(rd());
std::uniform_real_distribution<float> posWidth(100.0f, 1200.0f);
std::uniform_real_distribution<float> posHeight(100.0f, 700.0f);
std::uniform_real_distribution<float> velDist(-150.0f, 150.0f);


std::vector<Particle> particles;
for (int i = 0; i < numParticles; ++i) {
    Eigen::Vector2f startPos(posWidth(gen), posHeight(gen));
    Eigen::Vector2f startVel(velDist(gen), velDist(gen));

    // Fixed values: Mass 40, Color Red
    particles.emplace_back(40.0f, sf::Color::Red, startPos, startVel);
}


sf::Clock clock;

sf::Time totalTime = sf::Time::Zero; // Track total elapsed time


while (window.isOpen()) {
    sf::Time dtTime = clock.restart();
    float dt = dtTime.asSeconds();
    totalTime += dtTime;

    // Check for 5-minute timeout (300 seconds)
    if (totalTime.asSeconds() >= 300.0f) {
        std::cout << "5-minute timeout reached. Terminating program." << std::endl;
    }
}

```

```

        window.close();

        break;
    }

    while (const std::optional event = window.pollEvent()) {
        if (event->is<sf::Event::Closed>())
            window.close();
    }

    window.clear(sf::Color::Black);

    for (auto& p : particles) {
        p.update(dt);
        window.draw(p.shape);
    }

    window.display();
}

return 0;
}

```

Features of this build:

- **Timeout Logic:** Uses `sf::Time` to accumulate the duration of the simulation and closes the window once 300 seconds have passed.
- **Uniform Particles:** All particles are assigned a mass of **40** and the color **Red**.
- **Constraint Enforcement:** The user-entered particle count is capped at **100**.

- **Eigen Integration:** Particle positions and velocities are managed via `Eigen::Vector2f`, allowing for clean vector addition in the update step.

Would you like me to add an on-screen timer so you can see how much of the 5 minutes is remaining?

You said

power function for eigen library

Gemini said

To apply a power function to vectors or matrices in the **Eigen** library, you typically use the `.array()` coefficient-wise operations or the `pow()` function.

Coefficient-wise Power

In Eigen, standard Matrix objects (like `Eigen::Vector2f`) perform linear algebra operations by default. To raise every individual element in a vector to a power, you must first transition to an **Array** context using `.array()`.

C++

```
#include <Eigen/Core>
```

```
#include <iostream>
```

```
#include <cmath>
```

```
int main() {
```

```
    Eigen::Vector2f v(3.0f, 4.0f);
```

```
    // Squares each element: (3^2, 4^2) = (9, 16)
```

```
    Eigen::Vector2f v_squared = v.array().pow(2.0f);
```

```
    // Alternatively, using the global pow function with arrays
```

```
    Eigen::Vector2f v_cubed = Eigen::pow(v.array(), 3.0f);
```

```

std::cout << "Original: " << v.transpose() << std::endl;

std::cout << "Squared: " << v_squared.transpose() << std::endl;


return 0;

}

```

Integration into your Particle Simulation

Since your simulation uses `Eigen::Vector2f` for position and velocity, you can use these functions if you decide to implement physics like **Drag** or **Inverse-Square Law Gravitation**:

- **Air Resistance:** Drag is often proportional to the square of velocity.

C++

// Example: velocity-squared drag calculation

```
Eigen::Vector2f drag = -0.01f * velocity.array().pow(2.0f).matrix();
```

- **Distance Calculations:** You already use `.norm()` for magnitude in your demo, but for performance in gravity calculations, you might use `.squaredNorm()` to avoid the expensive square root operation.

Summary of Power Operations

Operation	Eigen Syntax	Description
Square Root	<code>v.array().sqrt()</code>	Square root of each element.
General Power	<code>v.array().pow(n)</code>	Raises each element to the power of <code>n</code> .
Square	<code>v.array().square()</code>	Faster specialized function for <code>.pow(2)</code> .
Cube	<code>v.array().cube()</code>	Faster specialized function for <code>.pow(3)</code> .

Would you like me to show you how to use these power functions to implement a "Black Hole" gravity effect for your red particles?

Gemini is AI and can make mistakes.